

Observability PR 第一性原理報告

日期：2026-06-01

範圍：PR #77 到 PR #81

repo：CloudNative_EventTicketSystem

1. Executive Summary

這一組 PR 的共同目標，不是把 observability 工具堆進產品 runtime，而是把 **Observability.pptx** 中尚未被系統表達的觀念，以最小、可驗證、reference-only 的方式補進 repo。它們都遵守同一個工程邊界：不改 app / worker 的內部邏輯、不改產品 API、不新增 production backing service，也不讓 metrics、logs、traces 或 alerts 變成 booking、ticket、check-in、reporting 或 audit 的真相來源。

從第一性原理看，observability 的價值不是「部署更多工具」，而是讓操作者能回答三個問題：

1. 使用者感受到什麼症狀？
2. 系統內部可能的原因在哪裡？
3. 這個信號是否足以支持下一個操作決策，而不破壞產品真相來源？

PR #77-#81 都在補第三層能力：把簡報提到的工具、格式、路由、儲存與 exporter 型態變成可審查的部署參考，並用測試保證它們停留在 reference surface，而不是悄悄進入 active Compose 或產品行為。

2. First Principles

Principle A: Observability is an evidence plane, not a control plane

觀測資料可以幫助調查與決策，但不能決定訂票、核銷、稽核或報表真相。CETS 的最終真相仍是 PostgreSQL-backed application contracts。這組 PR 因此只新增 Kubernetes observability reference manifests、README、architecture/spec 說明與測試，不把新元件掛到 `services/api/deploy/compose.yaml`。

Principle B: Preserve product behavior while expanding operational vocabulary

簡報提到 Logstash、OpenSearch、Thanos、Cortex、Grafana Agent、PagerDuty integration、mysql_exporter、logfmt、Common Log Format 等概念。直接接入這些工具會改變 runtime 或引入新的運維依賴；reference-only 實作則可以讓 repo 表達這些架構選項，同時保留現有產品行為。

Principle C: Each slice must be independently reviewable

每個 PR 只補一個可驗證缺口，且都包含：

- 一個新的 reference manifest。
- README.md 中的索引說明。
- docs/ARCHITECTURE.md 與 docs/specs/phase2-ws2-load-observability.md 的範圍更新。
- deploy tests，確認該 reference 存在、能 parse、且不接入 active runtime。
- local verification 與 act push。

Principle D: Tool coverage must map to a real deck concept

這組 PR 沒有任意加工具。每個切片都對應到簡報明確出現的 feature：

- log analysis tools : Logstash / OpenSearch ◦
- long-term metrics storage : Thanos / Cortex / Grafana Agent remote write ◦
- alerting ecosystem : Alertmanager / PagerDuty integration ◦
- metrics exporters : mysql_exporter ◦
- log formats : unstructured / semi-structured logfmt / Common Log Format / structured JSON ◦

3. PR-by-PR Analysis

PR #77 - Log analysis alternatives reference

URL : <https://github.com/gilbert12tw/Event-Ticket-System/pull/77>

Branch : **feature/cor-40-log-analysis-alternatives-reference**

Base : **feature/cor-40-service-level-reference**

Commit : **75ef291 feat(observability): add log analysis alternatives reference**

State : open draft, CLEAN

First-principles intent : production-scale logging needs centralized analysis, not only stdout or local file search. The deck names Logstash and OpenSearch-class tooling as part of the log ecosystem. The repo already had Loki / PLG and EFK references; this PR adds the Logstash + OpenSearch alternative without changing app logging.

What it added :

- **log-analysis-alternatives.yaml** with a Logstash HTTP JSON ingestion pipeline.
- OpenSearch StatefulSet and Service with persistent storage.
- README/spec/architecture references.
- A deploy test asserting Logstash, OpenSearch, JSON codec, index naming, StatefulSet, PVC, and indexed search coverage.

What it deliberately did not do :

- It did not change app or worker logger behavior.
- It did not route current stdout logs through Logstash.
- It did not make OpenSearch a product dependency.

Verification evidence :

- Image tags were checked for Logstash and OpenSearch.
- **go test ./deploy, go test ./internal/architecture**, Compose config, **git diff --check**, and **act push** passed.

PR #78 - Metrics storage alternatives reference

URL : <https://github.com/gilbert12tw/Event-Ticket-System/pull/78>

Branch : **feature/cor-40-metrics-storage-alternatives-reference**

Base : **feature/cor-40-log-analysis-alternatives-reference**

Commit : **078dabc feat(observability): add metrics storage alternatives reference**

State : open draft, CLEAN

First-principles intent : Prometheus local TSDB is a short-term store; long-term trend analysis needs a remote-write or durable storage pattern. The active Compose profile already had VictoriaMetrics, so this PR adds the other deck alternatives as reference only.

What it added :

- **metrics-storage-alternatives.yaml**.
- Thanos Receive StatefulSet and Service.
- Cortex StatefulSet and Service.
- Grafana Agent remote-write Deployment and Service.
- Prometheus and Grafana Agent remote-write examples pointing at Thanos and Cortex.

What it deliberately did not do :

- It did not replace VictoriaMetrics.
- It did not alter Prometheus scrape behavior in active Compose.
- It did not add long-term storage settings to app or worker.

Verification evidence :

- Image manifests were checked for Thanos, Cortex, and Grafana Agent.
- Deploy and architecture tests passed.
- Compose config and **git diff --check** passed.
- **act push** passed.

PR #79 - Alert routing reference

URL : <https://github.com/gilbert12tw/Event-Ticket-System/pull/79>

Branch : **feature/cor-40-paging-routing-reference**

Base : **feature/cor-40-metrics-storage-alternatives-reference**

Commit : **8246d36 feat(observability): add alert routing reference**

State : open draft, CLEAN

First-principles intent : alerts are useful only when they route the right class of symptom to the right operator channel. The active local profile only routes to a local review receiver. The deck mentions AlertManager and PagerDuty integration, so this PR documents that shape without activating paging.

What it added :

- **alert-routing-reference.yaml**.
- Alertmanager route tree with **local-review** and **pagerduty-on-call-reference** receivers.
- Critical severity route and inhibition example.
- Deployment and Service for reference Alertmanager.

What it deliberately did not do :

- It did not add PagerDuty credentials.
- It did not add production paging.
- It did not make Alertmanager a product dependency.

Verification evidence :

- Alertmanager image manifest was checked.
- Deploy and architecture tests passed.
- Compose config and `git diff --check` passed.
- `act push` passed.

PR #80 - Database exporter reference

URL : <https://github.com/gilbert12tw/Event-Ticket-System/pull/80>

Branch : `feature/cor-40-database-exporter-reference`

Base : `feature/cor-40-paging-routing-reference`

Commit : `4e487d2 feat(observability): add database exporter reference`

State : open draft, CLEAN

First-principles intent : exporters translate infrastructure or legacy systems into Prometheus metrics. The deck names `mysql_exporter`; CETS uses PostgreSQL, so the correct implementation is not to attach MySQL to the product runtime. The correct minimum is a reference for an external MySQL-compatible store.

What it added :

- `database-exporter-alternatives.yaml`.
- MySQL exporter Deployment and Service using `prom/mysqld-exporter:v0.17.2`.
- Prometheus scrape example with `signal_scope: database-exporter`.
- README/spec/architecture coverage that frames this as external to current CETS runtime.

What it deliberately did not do :

- It did not add MySQL to Compose.
- It did not change PostgreSQL-backed product truth.
- It did not introduce database exporter credentials into product config.

Verification evidence :

- MySQL exporter image manifest was checked.
- Deploy and architecture tests passed.
- Compose config and `git diff --check` passed.
- `act push` passed.

PR #81 - Log format reference

URL : <https://github.com/gilbert12tw/Event-Ticket-System/pull/81>

Branch : `feature/cor-40-log-format-reference`

Base : `feature/cor-40-database-exporter-reference`

Commit : `e8f203a feat(observability): add log format reference`

State : open draft, CLEAN

First-principles intent : log usefulness depends on structure. The deck distinguishes unstructured logs, semi-structured formats such as logfmt and Common Log Format, and structured JSON. CETS already emits structured JSON stdout logs; this PR records parser shapes for the full taxonomy without changing the app logger.

What it added :

- **log-format-reference.yaml**.
- Fluent Bit parser examples for structured JSON, logfmt, Common Log Format, and unstructured fallback.
- Sample log lines that show the practical distinction between formats.
- A new focused test file, **kubernetes_observability_log_reference_test.go**, to avoid pushing the existing reference test file over the 500-line limit.

What it deliberately did not do :

- It did not change **slog** setup.
- It did not add file logging.
- It did not add a remote logging SDK to the app.

Verification evidence :

- Deploy and architecture tests passed.
- Compose config and **git diff --check** passed.
- First **act push** hit a transient Docker Hub **500** while resolving **postgres:16.13-alpine**; after manifest lookup, rerunning the same command succeeded. This was recorded in memory as an **act** pitfall.

4. Cross-PR Pattern

The implementation pattern is consistent:

1. Identify a deck concept that is missing from the repo.
2. Decide whether the concept belongs in active runtime or future-platform reference.
3. If it would change behavior, implement it as reference-only.
4. Add tests that assert the reference exists and stays outside active deployment.
5. Update docs so reviewers understand why the reference exists.
6. Verify locally and with **act push**.

This pattern gives the repo observability vocabulary without silently expanding production scope.

5. What These PRs Prove

They prove that the repo now has reviewable examples for five previously weak or missing areas:

- Alternative centralized log analysis with Logstash and OpenSearch.
- Long-term metrics storage alternatives beyond the already-active VictoriaMetrics demo.
- Alert routing shape for local review and future PagerDuty-style on-call integration.
- Database exporter example for MySQL-compatible systems without changing CETS' PostgreSQL product architecture.
- Log format parser taxonomy matching the deck's structured / semi-structured / unstructured discussion.

They also prove the negative space:

- None of these references are loaded by active Compose.
- None add production backing services.

- None change app/worker code paths.
- None make observability data authoritative for product correctness.

6. Residual Risks and Review Focus

The main review risks are not product behavior regressions; the code intentionally avoids those. The review focus should be:

- Confirm reference manifests remain realistic enough to teach the intended architecture.
- Confirm future maintainers do not mistake reference-only YAML for deployable production manifests.
- Confirm test coverage continues to guard against reference files leaking product runtime config.
- Confirm stacked PR ordering is preserved, because each PR bases on the previous reference slice.
- Confirm binary/generated report artifacts, if committed, are expected for documentation purposes.

7. Evidence Appendix

PR stack:

PR	Topic	Branch	Base	Status
#77	Logstash + OpenSearch log analysis alternative	feature/cor-40-log-analysis-alternatives-reference	feature/cor-40-service-level-reference	open draft, CLEAN
#78	Thanos / Cortex / Grafana Agent metrics storage alternatives	feature/cor-40-metrics-storage-alternatives-reference	feature/cor-40-log-analysis-alternatives-reference	open draft, CLEAN
#79	Alertmanager + PagerDuty-style routing	feature/cor-40-paging-routing-reference	feature/cor-40-metrics-storage-alternatives-reference	open draft, CLEAN
#80	MySQL exporter database exporter reference	feature/cor-40-database-exporter-reference	feature/cor-40-paging-routing-reference	open draft, CLEAN
#81	Log format parser taxonomy	feature/cor-40-log-format-reference	feature/cor-40-database-exporter-reference	open draft, CLEAN

Common verification commands cited across the PRs:

- `go test ./deploy ...`
- `go test ./internal/architecture ...`
- `docker compose --env-file services/api/deploy/.env.example -f services/api/deploy/compose.yaml config`
- `git diff --check`
- `act push -P ubuntu-latest=ghcr.io/catthehacker/ubuntu:act-latest`

Conclusion：這組 PR 的核心成果，是把 Observability 簡報中的多個工具與概念轉成可版本化、可測試、可審查的 reference architecture，同時維持 CETS 產品行為與 correctness boundary 不變。